CCP6224 - OOAD Project

Trimester 2310

by CODE WARRIORS

Team Leader		
Lau Kaixuan	011-64442810	1221103162@student.mmu.edu.my
Team Members		
Iusupov Alimzhan	013-2963272	1231301318@student.mmu.edu.my
Wong Yong Kit	017-3538188	wong.yong.kit@student.mmu.edu.my
Wong Cii Voon	010-9003292	Wong.cii.voon@student.mmu.edu.my

Table of Contents

Project Description ***3***

Project Overview **3**

Objective **5**

Goals **6**

Compile and Run Instructions **7**

Prerequisites 7

Compilation and Execution Instructions 8

UML ***11***

UML Class Diagram **11**

Use Case Diagram **13**

Sequence Diagram **14**

Start Game 14

Play Move 15

Save History 16

Restart Game 17

Design Patterns ***18***

Implementation ***20***

User Documentation ***22***

Introduction **22**

Getting Started 22

Playing the Game 22

Game Controls 22

Saving the Game History 23

Restarting the Game 23

Troubleshooting 23

Contact Support ***23***

Project Description

Project Overview

Kwazam Chess game is an innovative Java-based chess variant that reimagines traditional chess with custom rules, dynamic piece behaviors, and strategic depth. Set on a 5x8 board, the game introduces five unique piece types—**RAM**, **TOR**, **BIZ**, **XOR**, and **SAU**—each with specialized movement mechanics. For example, the **RAM** piece reverses direction upon reaching the board's edge, simulating a "bounce" effect, while the **TOR** (a rook-like piece) and **XOR** (a bishop-like piece) morph into each other every two turns, requiring players to adapt their strategies continuously. The **BIZ** piece moves in non-traditional knight-like patterns, and the **SAU** acts as the game's "king," with its capture triggering an immediate victory.

The application leverages **Swing** to create an interactive graphical interface, featuring a rotatable board that flips perspectives after each move, ensuring both players engage from their strategic vantage points. The GUI integrates real-time gameplay elements such as a turn indicator, a timer tracking elapsed time, and visual highlights for valid moves, enhancing user immersion. Players can save their move history to a text file, restart the game at any point, or toggle full-screen mode for an optimized viewing experience.

Underlying the interface is a robust **Model-View-Controller (MVC)** architecture: the **ChessGameModel** manages game logic and state (e.g., turn rotation, piece transformations), the **ChessBoardView** renders the board and pieces dynamically, and the **KwazamChess** class acts as the controller, handling user inputs like mouse clicks and keyboard shortcuts (e.g., F11 for full-screen). Advanced features include automatic logging of moves in standardized notation, ensuring transparency and enabling post-game analysis.

The game emphasizes tactical complexity through mechanics like piece transformation (TOR/XOR), RAM direction reversal, and SAU vulnerability, encouraging players to anticipate opponents' moves while managing their own evolving board state. Error handling and usability are prioritized, with dialogs notifying players of game-over conditions or successful history saves. Built with object-oriented principles, the codebase modularizes components—such as piece

classes, event listeners, and file I/O operations—to support future extensibility, such as adding new piece types or adjusting movement rules. Kwazam Chess balances innovation with accessibility, offering both casual and strategic players a fresh take on chess through its blend of unconventional mechanics and polished interface design.

Objective

The primary objectives of KwazamChess are to:

1. **Implement Custom Chess Mechanics**: Encode unique movement rules for each piece type (e.g., RAM direction reversal, TOR/XOR transformations) and enforce turn-based logic to replicate the game's strategic depth programmatically.
2. **Deliver an Intuitive GUI**: Utilize Swing components to create a responsive interface with features like board rotation, move highlighting, and real-time status updates (e.g., turn labels, timer).
3. **Ensure Robust State Management**: Track game progress through the ChessGameModel, including valid move calculations, piece positions, and victory conditions (SAU capture).
4. **Enhance User Experience**: Integrate auxiliary tools such as a move history logger, full-screen mode, and restart functionality to streamline gameplay and accessibility.
5. **Demonstrate Software Design Best Practices**: Structure the codebase using MVC architecture to decouple logic, view, and control components, ensuring maintainability and scalability.

Secondary goals include validating user inputs to prevent invalid moves, providing visual feedback through highlighted squares and piece animations, and optimizing performance to handle real-time updates seamlessly. The project also aims to serve as an educational tool, illustrating object-oriented programming concepts like encapsulation (e.g., piece classes managing their own state), inheritance (Swing UI hierarchy), and polymorphism (dynamic method dispatching for piece movements).

Goals

KwazamChess aims to redefine chess as a dynamic and adaptive strategy game by merging traditional principles with innovative mechanics. Its overarching goals are to:

1. **Foster Strategic Thinking**: Challenge players to master evolving piece behaviors, such as TOR/XOR transformations and RAM reversals, which demand foresight and adaptability.
2. **Showcase Modular Software Design**: Highlight the effectiveness of MVC architecture and OOP principles in building complex applications, with clear separation between game logic (ChessGameModel), rendering (ChessBoardView), and user interaction (KwazamChess controller).
3. **Prioritize User-Centric Design**: Offer a polished, accessible interface with features like board rotation, full-screen mode, and save/restart options to cater to diverse player preferences.
4. **Encourage Extensibility**: Design a codebase that allows future developers to easily introduce new features, such as additional piece types, multiplayer support, or AI opponents, without disrupting core functionality.
5. **Promote Engagement**: Combine visual appeal (e.g., piece icons, color-coded boards) with interactive elements (e.g., move validation, game-over dialogs) to create an immersive experience for both novice and seasoned players.

By achieving these goals, KwazamChess not only delivers a novel gaming experience but also serves as a testament to effective software engineering practices, demonstrating how creativity and technical rigor can coalesce to transform a classic game into a modern strategic adventure.

Compile and Run Instructions

Instructions on how to compile and execute your program from the command line, with specific considerations for various operating systems (Windows, Linux, Mac).

Prerequisites

- **Java Development Kit (JDK) higher than version 1.8.0**: Ensure that JDK is installed on your system. The JDK program can be download from the [official Oracle website]() or install it using package manager (e.g., using homebrew or using sudo command in terminal install openjdk-11-jdk for MacOS or Linux Only). (For windows tools such as Chocolatey that are like homebrew)
- **IDE being use must support java (Only for people who using IDE only)**

Things need to be done before compile the program:

1. Change the image path to the absolute path that include in the program which is at the class **ChessPiece** (else the chess piece will be replace by symbol)

```
1 usage
public void draw(Graphics g, int x, int y, int size) {
    String imagePath;
    switch (type) {
        case SAU:
            imagePath = isWhite ? "/Users/user/Desktop/CCP6224-OOAD/Assignment/Project/Codes/
            break;
        case XOR:
            imagePath = isWhite ? "/Users/user/Desktop/CCP6224-OOAD/Assignment/Project/Codes/
            break;
        case TOR:
            imagePath = isWhite ? "/Users/user/Desktop/CCP6224-OOAD/Assignment/Project/Codes/
            break;
        case BIZ:
            imagePath = isWhite ? "/Users/user/Desktop/CCP6224-OOAD/Assignment/Project/Codes/
            break;
        case RAM:
            if(isRamTransformed()) {
                imagePath = isWhite ? "/Users/user/Desktop/CCP6224-OOAD/Assignment/Project/Co
            } else {
                imagePath = isWhite ? "/Users/user/Desktop/CCP6224-OOAD/Assignment/Project/Co
            }
            ;
            break;
        default:
            imagePath = null;
    }
```

Compilation and Execution Instructions

Using Command Prompt (Windows) or Terminal (MacOS / Linux)

1. **Open Command Line Interface**:
    a. **Windows**: Open Command Prompt.
    b. **MacOS / Linux**: Open Terminal.

2. **Navigate to the program directory in the command prompt / Terminal where you place it**: Use "cd" command to navigate to the directory containing the KwazamChess source files. For example:

    ```
    cd "/absolute path to KwazamChess directory"
    ```

3. **Compile the Java Source Files**: Compile the program using `javac` command:

    ```
    javac KwazamChess.java
    ```

    This will compile the KwazamChess.java file and any other related files

4. **Run the Program**: After successful compilation, use the `java` command to run the program:

    ```
    java KwazamChess
    ```

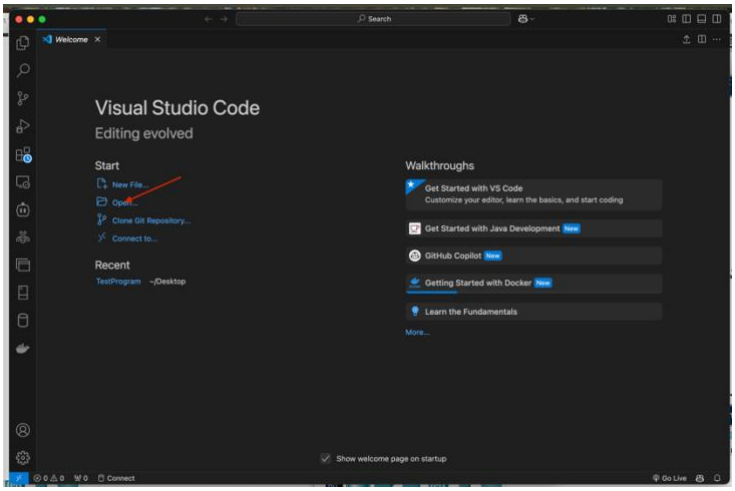
    This command will start the KwazamChess application.

Using Compiler

For Visual Studio Code, Visual Studio 2022, and IntelliJ IDEA

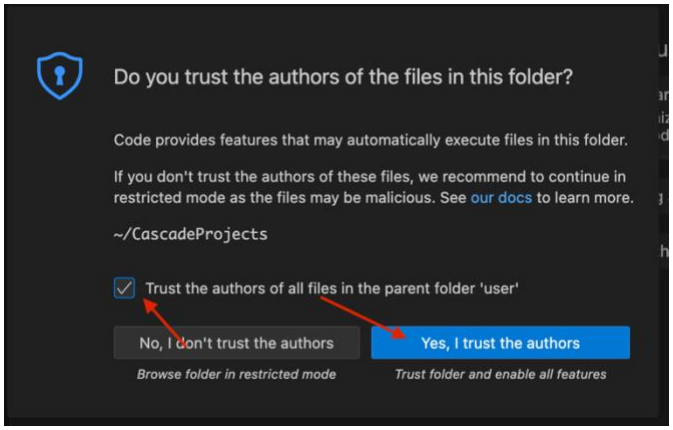
Important: IntelliJ IDEA user need to add or edit the run / debug configuration before running the code

1. Open the Visual Studio application
2. Click on the open option



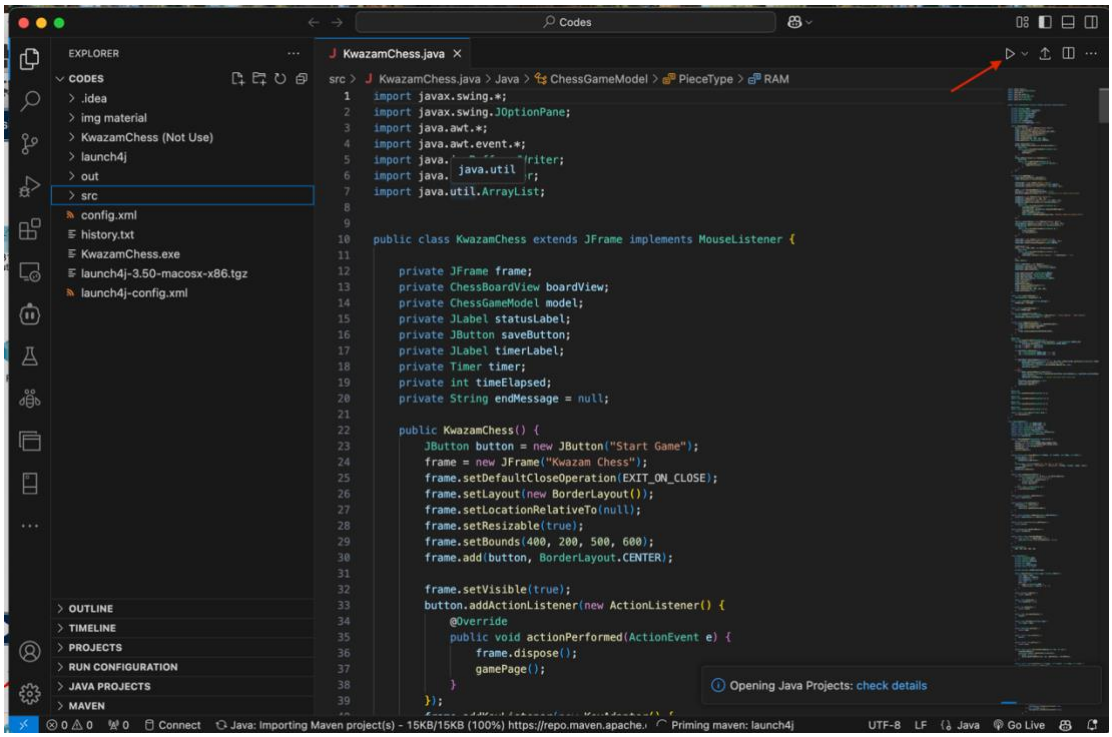
3. Choose the directory that you place the program then open the program directory

4. Click on the trust the author of all files in the .... option and click yes, I trust the author option.



5. Double click the KwazamChess.java

6. Click on the run button



7. Then the program might run successfully

UML

UML Class Diagram

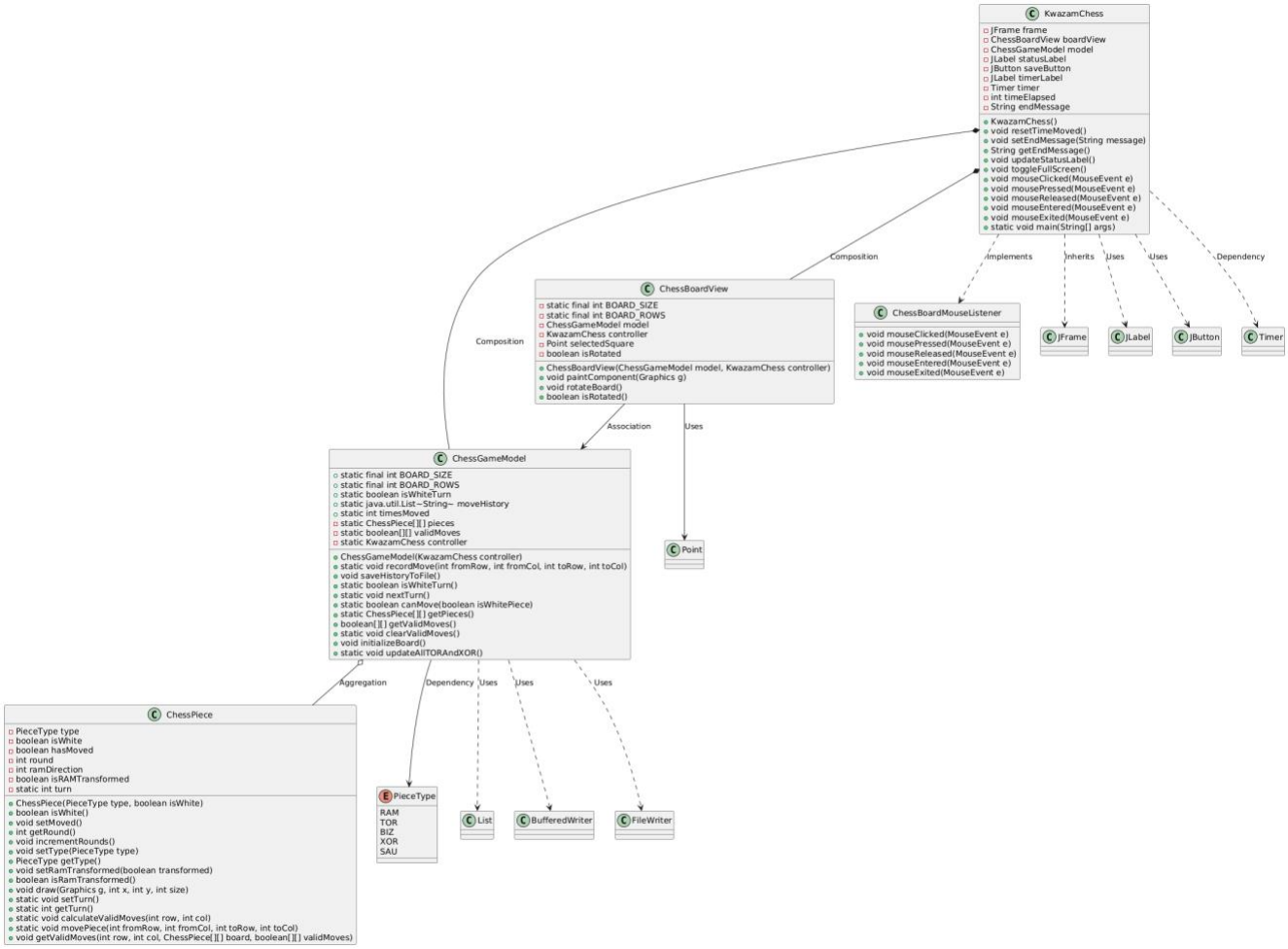
The KwazamChess UML class diagram follows a Model-View-Controller (MVC) architecture to separate game logic, rendering, and user interaction. The **KwazamChess** class acts as the controller, initializing the GUI and handling input events like mouse clicks and keyboard shortcuts. The **ChessGameModel** manages core gameplay mechanics, including piece movements, turn logic, and move history logging, while the renders the board and highlights valid moves. Nested within the model, the ChessPiece class defines piece-specific behaviors (e.g., RAM direction reversal, TOR or XOR transformations) and movement rules. Relationships between classes include aggregation (e.g., the controller holds the model and view), composition (the model contains a grid of **ChessPiece** objects), and dependencies (the view relies on the model for data). The design emphasizes modularity, enabling future enhancements like new piece types or expanded gameplay features.

Classes and Roles

1. **KwazamChess (Controller)**:
    - Manages GUI setup (buttons, labels, frames).
    - Handles mouse/keyboard events (e.g., piece selection, full-screen toggle).
    - Updates game state via ChessGameModel.

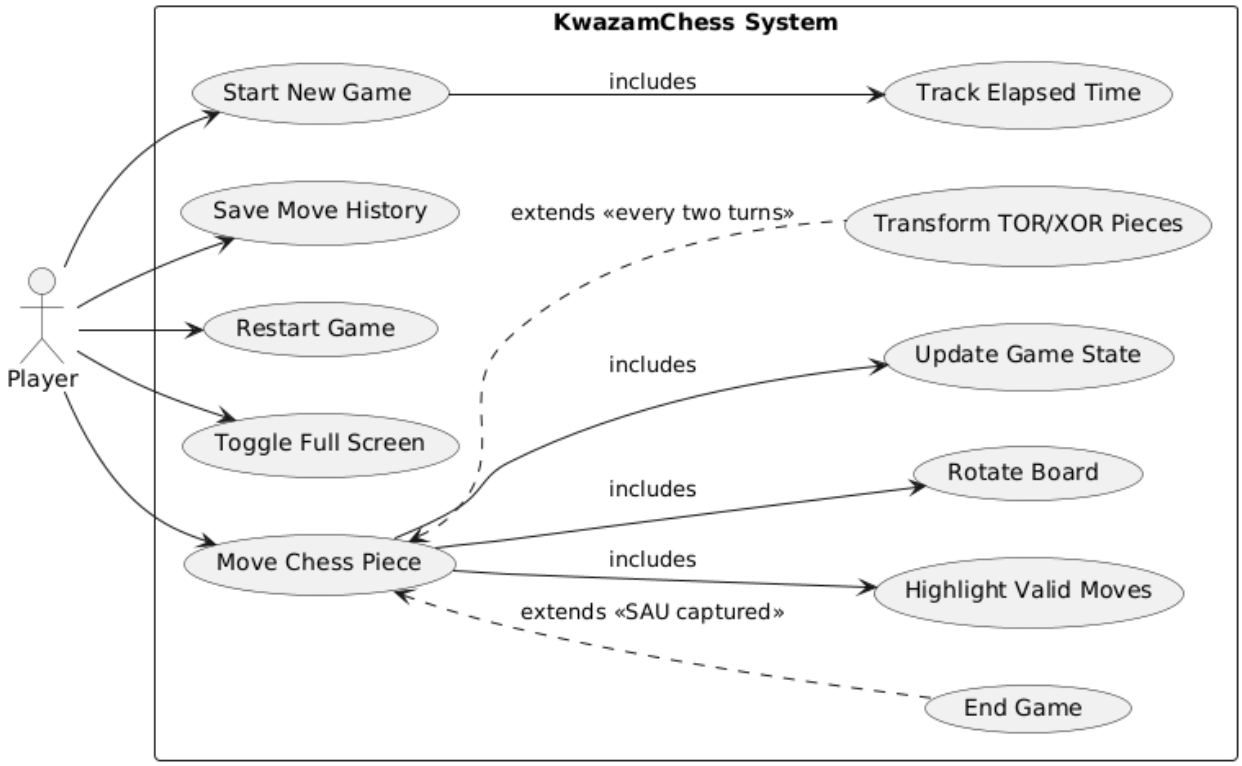
2. **ChessGameModel (Model)**:
    - Stores board state, valid moves, and turn logic.
    - Implements piece movement rules and transformations (e.g., TOR ↔ XOR every 2 turns).
    - Logs moves to history.txt.
3. **ChessBoardView (View)**:
    - Renders the board and pieces using Swing Graphics.
    - Rotates board perspective and highlights valid moves.
4. **ChessPiece (Nested Class)**:
    - Defines piece types (RAM, TOR, BIZ, XOR, SAU) and their movement logic.
    - Handles transformations and direction changes (e.g., RAM reversal at edges).

Diagram:


KwazamChess
- JFrame frame
- ChessBoardView boardView
- ChessGameModel model
- JLabel statusLabel
- JButton saveButton
- JLabel timerLabel
- Timer timer
- int timeElapsed
- String endMessage
+ KwazamChess()
+ void resetTimeMoved()
+ void setEndMessage(String message)
+ String getEndMessage()
+ void updateStatusLabel()
+ void toggleFullScreen()
+ void mouseClicked(MouseEvent e)
+ void mousePressed(MouseEvent e)
+ void mouseReleased(MouseEvent e)
+ void mouseEntered(MouseEvent e)
+ void mouseExited(MouseEvent e)
+ static void main(String[] args)
Composition
Composition
Implements
Inherits
Uses
Uses
Dependency
ChessBoardView
- static final int BOARD_SIZE
- static final int BOARD_ROWS
- ChessGameModel model
- KwazamChess controller
- Point selectedSquare
- boolean isRotated
+ ChessBoardView(ChessGameModel model, KwazamChess controller)
+ void paintComponent(Graphics g)
+ void rotateBoard()
+ boolean isRotated()
ChessBoardMouseListener
+ void mouseClicked(MouseEvent e)
+ void mousePressed(MouseEvent e)
+ void mouseReleased(MouseEvent e)
+ void mouseEntered(MouseEvent e)
+ void mouseExited(MouseEvent e)
JFrame
JLabel
JButton
Timer
Association
Uses
Point
ChessGameModel
- static final int BOARD_SIZE
- static final int BOARD_ROWS
- static boolean isWhiteTurn
- static java.util.List<String> moveHistory
- static int timesMoved
- static ChessPiece[][] pieces
- static boolean[][] validMoves
- static KwazamChess controller
+ ChessGameModel(KwazamChess controller)
+ static void recordMove(int fromRow, int fromCol, int toRow, int toCol)
+ void saveHistoryToFile()
+ static boolean isWhiteTurn()
+ static void nextTurn()
+ static boolean canMove(boolean isWhitePiece)
+ static ChessPiece[][] getPieces()
+ boolean[][] getValidMoves()
+ static void clearValidMoves()
+ void initializeBoard()
+ static void updateAllTORAndXOR()
Aggregation
Dependency
Uses
Uses
Uses
ChessPiece
- PieceType type
- boolean isWhite
- boolean hasMoved
- int round
- int ramDirection
- boolean isRAMTransformed
- static int turn
+ ChessPiece(PieceType type, boolean isWhite)
+ boolean isWhite()
+ void setMoved()
+ int getRound()
+ void incrementRounds()
+ void setType(PieceType type)
+ PieceType getType()
+ void setRamTransformed(boolean transformed)
+ boolean isRamTransformed()
+ void draw(Graphics g, int x, int y, int size)
+ static void setTurn()
+ static int getTurn()
+ static void calculateValidMoves(int row, int col)
+ static void movePiece(int fromRow, int fromCol, int toRow, int toCol)
+ void getValidMoves(int row, int col, ChessPiece[][] board, boolean[][] validMoves)
PieceType
RAM
TOR
BIZ
XOR
SAU
List
BufferedWriter
FileWriter

Use Case Diagram

Description: This diagram illustrates the main functions of the KwazamChess application from a user's perspective. The user can perform five primary actions: Start Game, Play Move, Save History, Restart Game, and Toggle Full Screen.



Sequence Diagram

Start Game

Description: This sequence diagram shows the interactions between the user and the KwazamChess application when the user starts the game. It outlines the process from the user clicking the "Start Game" button to initializing and displaying the game page.

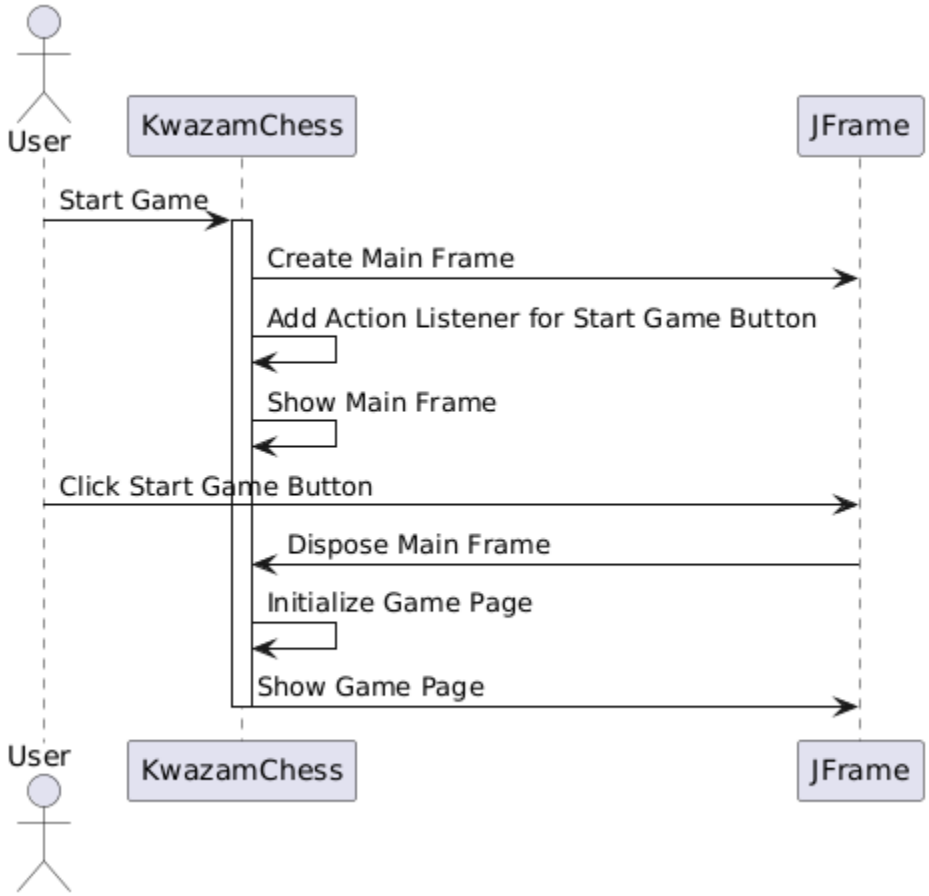


Figure 3

Play Move

Description: This sequence diagram details the interactions involved when the user plays a move. It demonstrates how the ChessBoardView and ChessGameModel handle the selection and movement of a chess piece, including updating the game state and switching turns.

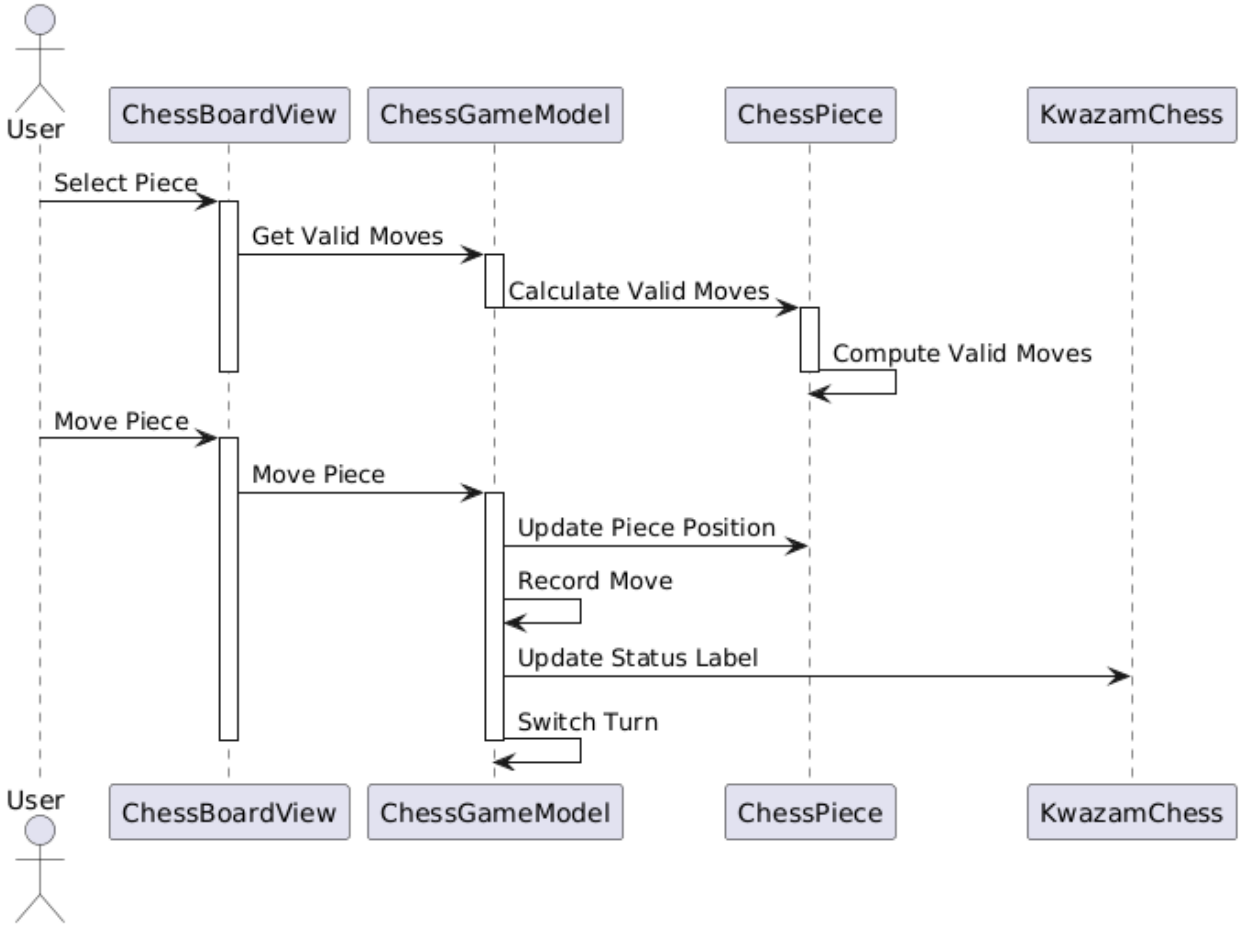


Figure 4

Save History

Description: This diagram illustrates the sequence of actions that occur when the user clicks the "Save History" button. It shows how the KwazamChess application instructs the ChessGameModel to save the move history to a file and then displays a confirmation message to the user.

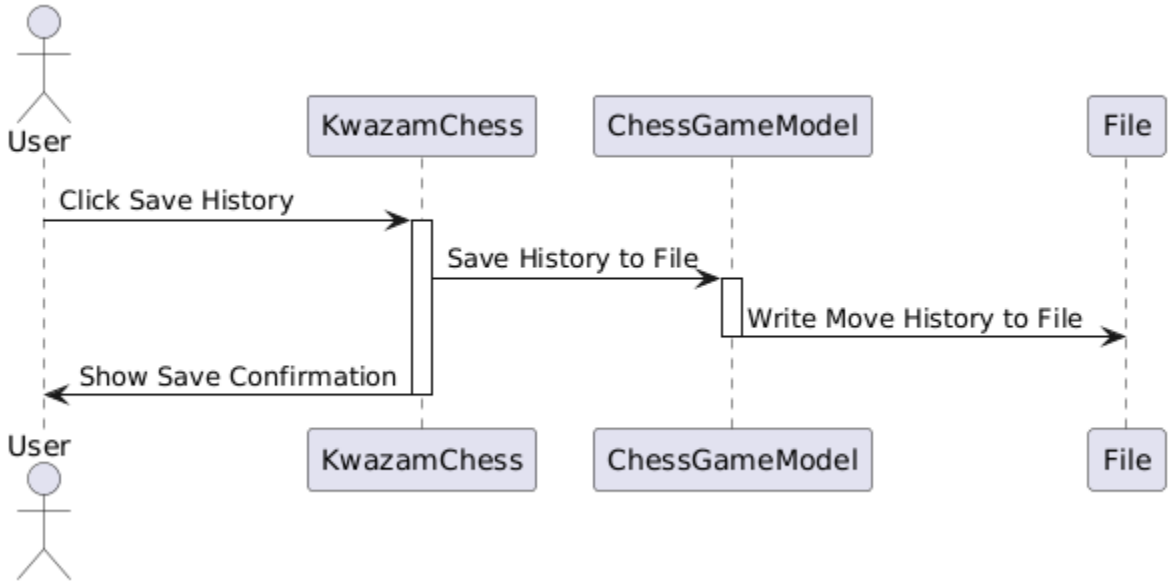


Figure 5

Restart Game

Description: This sequence diagram depicts the process of restarting the game. It shows the steps from the user clicking the "Restart Game" button to disposing of the current game and initializing a new one, followed by displaying the new game page.

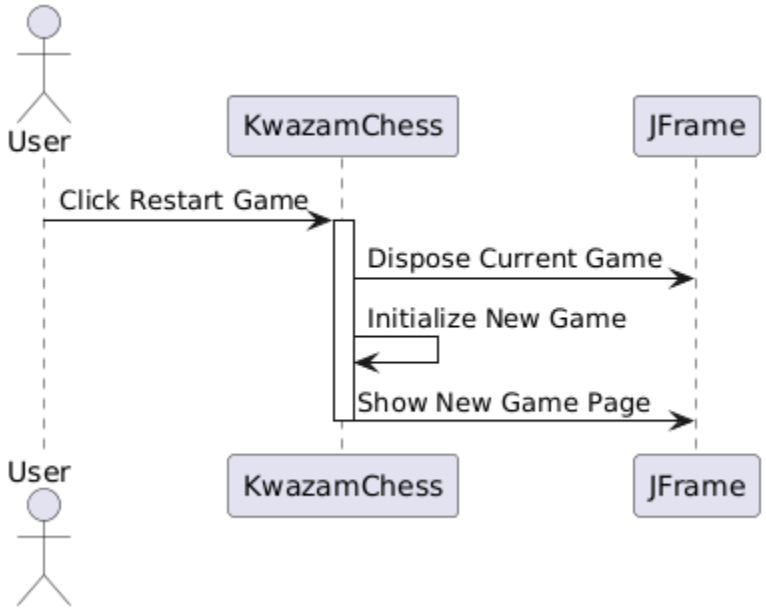


Figure 6

Design Patterns

- **Observer Pattern**: The KwazamChess class implements the Observer pattern by using Swing's event listeners (MouseListener and ActionListener). For example, the **mouseClicked()** method observes mouse events on the board to handle piece selection and movement, while **ActionListener** for buttons like "Save History" triggers actions such as saving move data. These listeners decouple user input (e.g., clicks) from the logic that updates the model (ChessGameModel) and refreshes the view (ChessBoardView).

- **State Pattern**: The nested ChessPiece class within **ChessGameModel** partially implements the State pattern. Each piece's behavior (e.g., movement logic) changes based on its **PieceType** (RAM, TOR, XOR, etc.). For example, RAM pieces reverse direction at board edges, while TOR or XOR pieces transform every two turns. However, state transitions are managed procedurally using switch statements in **getValidMoves()**, rather than encapsulating states in separate classes.

- **Template Pattern**: The ChessPiece class uses a structure resembling the Template Method pattern in its **getValidMoves()** method. This method defines a skeleton algorithm that delegates to type-specific logic (e.g., **getRAMMoves()**, **getTORMoves()**) via a switch statement. While not a formal Template Method, it hierarchically organizes movement rules for different piece types.

- **MVC (Model-View-Controller) Pattern**:
    - **Model**: ChessGameModel - Contains the game's data and logic (e.g., pieces, moves, turns).
    - **View**: ChessBoardView - Renders the game's user interface (1.g., board, pieces).
    - **Controller**: KwazamChess - Handles user input and updates the model and view accordingly.

- **Singleton Pattern**: The **ChessGameModel** class mimics a Singleton pattern through its static fields (e.g., pieces, isWhiteTurn, moveHistory). These static variables act as a global state, ensuring all components access the same board configuration, turn system, and move history. While not a strict Singleton, this design centralizes critical game data.

- **Factory Pattern**: The ChessPiece class's **draw()** method acts as a simple factory. It selects image paths or Unicode symbols based on the **PieceType** (e.g., SAU uses a king symbol, RAM uses directional images). This centralizes rendering logic, though it relies on conditional checks instead of a dedicated factory class.

- **Command Pattern**: Methods like **saveHistoryToFile()** in **ChessGameModel** and **toggleFullScreen()** in **KwazamChess** encapsulate actions triggered by UI events, loosely aligning with the Command pattern.

- **Composite Pattern**: The **ChessBoardView** class (a JPanel subclass) leverages Swing's inherent Composite pattern. It nests UI components (e.g., board squares, piece icons) in a hierarchical structure, allowing the panel to manage rendering and layout as a unified container.

Implementation

Code Structure

- **KwazamChess** - Main application class
- **ChessGameModel** - Game logic and state management
- **ChessBoardView** - GUI rendering and user interaction

Key Classes and Methods

- **KwazamChess (Main Application Class)**
  - **Constructor**: Initializes the game window and start screen.
  - **gamePage()**: Sets up the main game interface.
  - **updateStatusLabel()**: Updates the turn indicator.
  - **toggleFullScreen()**: Handles fullscreen functionality.
- **ChessGameModel (Game Logic)**
  - **Constants**:
    - `BOARD_SIZE = 5`
    - `BOARD_ROWS = 8`
  - **State Management**:
    - `pieces[][]`: Stores the game board state.
    - `validMoves[][]`: Tracks possible moves.
    - `isWhiteTurn`: Manages turn alternation.
  - **Key Methods**:
    - **recordMove()**: Logs game moves.
    - **saveHistoryToFile()**: Persists game history.
    - **canMove()**: Validates move legality.
- **ChessBoardView (GUI Component)**
  - Handles board rendering and piece visualization.
  - Manages mouse interactions for piece movement.
  - Implements board highlighting for selected pieces and valid moves.
  - **Special Features**
- **Custom Chess Pieces**
  - The game introduces unique chess pieces:
  - **RAM**
  - **TOR**
  - **BIZ**
  - **XOR**

- **SAU**

Game Features

- Move history tracking and saving.
- Timer functionality.
- Turn-based gameplay.
- Visual move validation.
- Game state persistence.

Software Design Approach

- **Model** (ChessGameModel): Handles game logic and state management.
- **View** (ChessBoardView): Responsible for rendering and user interface.
- **Controller** (KwazamChess): Manages application flow and event handling.
  -

User Documentation

Introduction

Welcome to KwazamChess, an engaging and innovative chess game application. This manual will guide you through the functionalities and features of the game, ensuring you have a seamless and enjoyable experience.

Getting Started

1. Launch the KwazamChess application.
2. A main window titled "Kwazam Chess" will appear with a "Start Game" button in the center.
3. Click the "Start Game" button to proceed to the game page.

Playing the Game

1. **Game Board**: The game board will be displayed with the initial setup of pieces.
2. **Status Label**: The status label at the top will indicate whose turn it is (White's or Black's).
3. **Timer**: The timer on the right side will show the elapsed time since the game started.
4. **Save History**: Click the "Save History" button to save the move history to a file named `history.txt`.
5. **Restart Game**: Click the "Restart Game" button to start a new game. The current game will be disposed of, and a new game board will be displayed.

Game Controls

1. **Select a Piece**: Click on a piece you want to move. Valid moves for the selected piece will be highlighted in green.
2. **Move a Piece**: Click on a valid destination square to move the selected piece there. The game will automatically switch turns after a move is made.
3. **Full Screen Mode**: Press F11 to toggle full-screen mode. Press F11 again to return to windowed mode.

Saving the Game History

1. During the game, you can save your move history by clicking the "Save History" button.
2. A confirmation message will appear indicating that the history has been saved to `history.txt`.

Restarting the Game

1. If you wish to restart the game, click the "Restart Game" button.
2. The current game will close, and a new game will start.

Troubleshooting

1. **Game Not Launching**: Ensure that Java is properly installed on your system. Check if you followed the installation and running steps correctly.
2. **Invalid Moves**: Make sure you are selecting valid moves for the piece. Valid moves are highlighted in green.
3. **Saving History**: If the history is not being saved, ensure that the application has permission to write to the directory.

Contact Support

For any issues or further assistance, please email our team member or team leader